

M13 / PROTECTED ANSWERS

M13 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 5

M13 | DELTA LAKE + LAKEHOUSE 5

EXPLAINED REVIEWS - OPEN AFTER ATTEMPT 5

Review 1 | DE13-01 - Data Lake vs Warehouse vs Lakehouse 5

Worked recovery 5

What a strong answer must prove 5

Debugging path 5

Changed retry - do this without looking back 6

Review 2 | DE13-02 - Why plain files are not tables 6

Worked recovery 6

What a strong answer must prove 6

Debugging path 6

Changed retry - do this without looking back 7

Review 3 | DE13-03 - Delta transaction log, ACID and table versions 7

Worked recovery 7

What a strong answer must prove 7

Debugging path 7

Changed retry - do this without looking back 7

Review 4 | DE13-04 - Create, read, update and delete 8

Worked recovery 8

What a strong answer must prove 8

Debugging path	8
Changed retry - do this without looking back	8
Review 5 DE13-05 - Schema enforcement and schema evolution	9
Worked recovery	9
What a strong answer must prove	9
Debugging path	9
Changed retry - do this without looking back	9
Review 6 DE13-06 - Time travel and version history	10
Worked recovery	10
What a strong answer must prove	10
Debugging path	10
Changed retry - do this without looking back	10
Review 7 DE13-07 - MERGE, upserts and idempotent incremental loads	11
Worked recovery	11
What a strong answer must prove	11
Debugging path	11
Changed retry - do this without looking back	11
Review 8 DE13-08 - Slowly Changing Dimension Type 2 with Delta	12
Worked recovery	12
What a strong answer must prove	12
Debugging path	12
Changed retry - do this without looking back	12
Review 9 DE13-09 - Bronze, Silver and Gold lakehouse layers	13

Worked recovery 13

What a strong answer must prove 13

Debugging path 13

Changed retry - do this without looking back 13

Review 10 | DE13-10 - Compaction, OPTIMIZE, VACUUM and maintenance boundaries 14

Worked recovery 14

What a strong answer must prove 14

Debugging path 14

Changed retry - do this without looking back 14

Review 11 | DE13-11 - Integrated lakehouse pipeline and debugging checkpoint 15

Worked recovery 15

What a strong answer must prove 15

Debugging path 15

Changed retry - do this without looking back 15

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

M13 | DELTA LAKE + LAKEHOUSE

EXPLAINED REVIEWS - OPEN AFTER ATTEMPT

Worked recovery and reasoning for every M13 lesson. These are not one-line answer keys; use them to diagnose a saved first attempt, then close the review and complete a fresh changed retry.

PROTECTED MATERIAL - open only after saving your closed-book attempt. Course runtime baseline: Spark 4.2.0 + Delta Lake 4.4.0. Live Delta/PySpark execution remains learner-PC proof unless explicitly recorded.

Review 1 | DE13-01 - Data Lake vs Warehouse vs Lakehouse

Worked recovery

Example: orders arrive as Parquet each hour. At 14:00 an upstream correction changes order O17. If consumers scan every file in a folder, they may see both old and corrected rows unless another convention tells them which one wins. A Delta table records a new committed version whose snapshot includes the files/actions representing the correction. Readers query the table snapshot, not an accidental folder listing. A reporting team says yesterday's revenue changed even though no report code changed. Storage contains the original order files plus a late correction file. Ask: what is the table contract? Can you identify one committed snapshot? Is duplicate/correction precedence encoded in a log/table operation, or is every reader inventing its own rule? The incident is not solved by buying faster storage; it is solved by clarifying state and commit semantics.

What a strong answer must prove

- Uses the lesson's business grain/key rather than only an API name.
- Separates physical files from logical table state and cites version/history when relevant.
- Includes an independent reconciliation, not just successful execution.
- States a realistic failure mode and the first evidence to inspect.
- Does not claim live Delta runtime if only static/simulator evidence exists.

Debugging path

A common beginner failure is to call any collection of Parquet files a lakehouse. Parquet is a columnar file format; it does not by itself provide a transaction log or table history. Another failure is saying a lakehouse 'replaces warehouses.' Real architectures may use both, and the deciding factors include workload, governance, latency, team skills, interoperability and cost. Recovery: Separate three questions in architecture reviews: where bytes live, how a table snapshot is defined, and which compute engine reads/writes it. Lakehouse thinking becomes clearer when those layers are not

collapsed into one vendor name. In this module, Spark is the compute engine, Parquet remains the underlying data-file format, and Delta adds table metadata/transaction semantics. That layered mental model will make later Iceberg/Hudi comparisons easier without needing to learn them now.

Changed retry - do this without looking back

Now change the scenario: the company has immutable compliance archives that are never updated. Explain why a simple lake zone may be completely appropriate for that zone even if the curated serving layer uses Delta tables.

Review 2 | DE13-02 - Why plain files are not tables

Worked recovery

Suppose version 0 adds files A and B. A correction creates version 1 that adds file C and logically removes A. The current snapshot is B + C, even if A still exists on disk for time-travel/retention reasons. A naive recursive Parquet reader might read A + B + C and produce wrong results. A Delta reader consults the table log and uses the active snapshot. A junior engineer compares storage size with current table size and says Delta duplicated data. History shows an overwrite and the old files still exist. Diagnose correctly: logical removal can precede physical cleanup so time travel and concurrent readers remain safe. Confirm current row/key totals through the Delta snapshot before discussing retention or VACUUM.

What a strong answer must prove

- Uses the lesson's business grain/key rather than only an API name.
- Separates physical files from logical table state and cites version/history when relevant.
- Includes an independent reconciliation, not just successful execution.
- States a realistic failure mode and the first evidence to inspect.
- Does not claim live Delta runtime if only static/simulator evidence exists.

Debugging path

Do not delete files manually because they 'look old.' Physical presence is not the same as logical membership. Do not use filesystem modified time as a business version. Do not write unrelated files inside a table directory and assume Delta will protect them from every maintenance operation. Table-managed directories need a clear ownership boundary. Recovery: The important abstraction is not 'Delta has JSON files.' It is that readers and writers agree on a protocol for constructing snapshots and committing changes. Checkpoints can compact log state for efficient reconstruction; the details evolve across versions, but the contract remains: table state comes from committed metadata, not folder intuition. This is why table formats matter for interoperability - independent engines can implement the protocol rather than reverse-engineer one application's filenames.

Changed retry - do this without looking back

Change version 2 so a failed job produced an unreferenced Parquet file E without a successful commit. Explain why 'file exists' is not enough to make E part of the table and why manual cleanup must still respect retention/ownership rules.

Review 3 | DE13-03 - Delta transaction log, ACID and table versions

Worked recovery

Suppose two writers both read version 5. Writer A updates partition/date X and commits version 6. Writer B then tries to commit based on its earlier snapshot. Whether B can safely commit depends on the actions/conflicts involved; the system must not silently combine incompatible changes into an unexplained partial state. Your debugging evidence is table history and conflict/error information, not 'both jobs finished their Python loops.' A pipeline wrote several Parquet files and then crashed before its intended final state. A raw file consumer sees some new files and some old ones. A Delta consumer still reads the latest committed snapshot. Investigate whether a commit was recorded, what version became current, and whether the crashed job should retry idempotently. The right question is not 'did some files get created?' but 'what table version committed?'

What a strong answer must prove

- Uses the lesson's business grain/key rather than only an API name.
- Separates physical files from logical table state and cites version/history when relevant.
- Includes an independent reconciliation, not just successful execution.
- States a realistic failure mode and the first evidence to inspect.
- Does not claim live Delta runtime if only static/simulator evidence exists.

Debugging path

Do not claim ACID means no job can fail. Jobs fail; ACID helps define what becomes committed and visible. Do not claim isolation means every writer is globally serialized from start to finish. Delta uses optimistic concurrency patterns. Do not equate schema enforcement with every business-quality rule - a string can be valid by schema and still contain an invalid customer code. Recovery: Table history is operational evidence. It can answer which operation created a state, when, and often metrics such as rows/files affected. That does not replace application-level audit columns or source reconciliation, but it gives you a reliable table-state timeline. In production incident response, combine three layers: source evidence, pipeline run evidence and table commit/history evidence. A clean Delta history cannot prove the source delivered the right business data.

Changed retry - do this without looking back

Change the scenario so two jobs touch disjoint date partitions. Explain why 'concurrent' does not automatically mean 'conflict,' but why you should still rely on actual commit behavior and not hard-code assumptions about every operation.

Review 4 | DE13-04 - Create, read, update and delete

Worked recovery

If UPDATE orders SET status='closed' WHERE order_id='02' targets one known key, you can prove scope before and after. If instead a developer writes WHERE status IS NOT NULL, the command may be syntactically valid and catastrophically broad. Engineering safety comes from previewing the predicate population and storing reconciliation controls, not from trusting the operation name. A delete job intended to remove 120 test records reports 1.8 million rows affected. Stop before maintenance cleanup. Preserve history, inspect the predicate and current/previous versions, and determine whether a restore or corrected write is appropriate. Do not run VACUUM during investigation because you may destroy the historical files needed for recovery.

What a strong answer must prove

- Uses the lesson's business grain/key rather than only an API name.
- Separates physical files from logical table state and cites version/history when relevant.
- Includes an independent reconciliation, not just successful execution.
- States a realistic failure mode and the first evidence to inspect.
- Does not claim live Delta runtime if only static/simulator evidence exists.

Debugging path

Never demonstrate UPDATE/DELETE on valuable data without a safe sandbox, predicate preview and recovery plan. Do not tell a privacy stakeholder that DELETE immediately erases historical storage; VACUUM/retention and governance policies matter. Avoid using overwrite as a casual substitute for a targeted correction when it increases blast radius. Recovery: DML correctness has two dimensions: selection correctness and resulting-state correctness. Selection correctness asks whether the predicate matched the intended population. Resulting-state correctness asks whether keys, grain, totals and downstream constraints remain valid after the new commit. Production runbooks should capture both, plus a rollback/restore decision path. Delta gives you table mechanics; it does not know your business invariant unless you encode/check it.

Changed retry - do this without looking back

Change the delete requirement from a specific key to 'all records older than a cutoff except legally retained customers.' Design the predicate preview and validation checks you would require before allowing execution.

Review 5 | DE13-05 - Schema enforcement and schema evolution

Worked recovery

Adding nullable `loyalty_tier` can be a compatible business change if downstream consumers tolerate it. Changing `customer_id` from a stable string identifier to an integer code is not 'just schema evolution'; it may change key semantics. A type-compatible technical write can still violate the business contract, so schema review must include meaning and downstream dependencies. A producer deploys a new `discount_code` field Friday evening. The Silver table starts failing. Your options include reject/quarantine until the contract is approved, evolve the schema with explicit consumer checks, or route the field only in Bronze while curated layers remain stable. The wrong response is to bypass enforcement without understanding which downstream jobs select *, serialize fixed schemas or calculate hashes from all columns.

What a strong answer must prove

- Uses the lesson's business grain/key rather than only an API name.
- Separates physical files from logical table state and cites version/history when relevant.
- Includes an independent reconciliation, not just successful execution.
- States a realistic failure mode and the first evidence to inspect.
- Does not claim live Delta runtime if only static/simulator evidence exists.

Debugging path

Do not globally enable auto-merge merely to eliminate schema errors. That can turn a useful failure signal into silent drift. Do not use `overwriteSchema` as a first response to one bad batch. Do not treat column presence as proof of valid data: a newly added field may be 100% null or populated with malformed values. Recovery: Schema governance should distinguish additive, breaking and semantic changes. Additive does not automatically mean harmless: column order assumptions, CSV exports, downstream strict schemas and wildcard selects can still break. For important tables, store schema-change evidence alongside deployment/runbook notes. Delta records table metadata changes, but team process decides whether the change was authorized.

Changed retry - do this without looking back

Change the request to rename a business-critical column used by dashboards. Explain why this is a different risk class from adding a nullable field and how you would stage compatibility rather than flipping it instantly.

Review 6 | DE13-06 - Time travel and version history

Worked recovery

A finance report generated from table version 42 can be reproduced later by reading version 42 - if its required files/history are still retained. If the team only records 'report ran Tuesday' and uses latest data on rerun, the result may differ because late-arriving corrections created newer versions. Persisting relevant version identifiers in job/report metadata dramatically improves reproducibility. After an accidental delete, an engineer wants to VACUUM first because storage looks messy. Stop. Inspect history and verify whether a pre-delete version can still be read. If recovery is required, choose an explicit restore/rewrite path, reconcile the recovered state and only then revisit retention cleanup. Maintenance must not destroy your recovery evidence during the incident.

What a strong answer must prove

- Uses the lesson's business grain/key rather than only an API name.
- Separates physical files from logical table state and cites version/history when relevant.
- Includes an independent reconciliation, not just successful execution.
- States a realistic failure mode and the first evidence to inspect.
- Does not claim live Delta runtime if only static/simulator evidence exists.

Debugging path

Do not promise time travel forever. Do not run VACUUM with aggressively low retention just to save local disk and then expect old versions to work. Do not confuse reading an old version with restoring it: a historical read is non-destructive; RESTORE creates a new current commit and can affect downstream incremental consumers. Recovery: Version IDs are better operational anchors than wall-clock guesses because clocks, time zones and delayed commits complicate timestamp reasoning. Still, timestamp travel can be useful when business processes record times rather than versions. For robust pipelines, capture the input table version(s), output version and job/run ID together. That forms a lineage breadcrumb even before a full metadata platform is introduced.

Changed retry - do this without looking back

Assume the table's historical files older than seven days have been vacuumed. Explain what evidence may remain, what data may no longer be readable, and why a separate backup/compliance strategy can still be necessary.

Review 7 | DE13-07 - MERGE, upserts and idempotent incremental loads

Worked recovery

Target O2 has updated_at=10:00, amount 80. Source O2 arrives with updated_at=10:05, amount 85, so it may supersede. If a delayed source O2 later arrives with updated_at=09:55, a naive matched-update would roll the table backward. Add a matched condition requiring the source update to be newer. Then reconcile that the older late batch caused no business regression. A nightly job is retried after the orchestrator times out. The first attempt may actually have committed. The second attempt reprocesses the same source batch. An append-only target doubles revenue; an idempotent MERGE target should converge to the same key state. Investigate source batch identity, target history and reconciliation before deciding whether to skip or rerun.

What a strong answer must prove

- Uses the lesson's business grain/key rather than only an API name.
- Separates physical files from logical table state and cites version/history when relevant.
- Includes an independent reconciliation, not just successful execution.
- States a realistic failure mode and the first evidence to inspect.
- Does not claim live Delta runtime if only static/simulator evidence exists.

Debugging path

Do not feed duplicate source keys into MERGE and assume Delta will choose the latest row for you. Ambiguous multiple source matches can fail or create nondeterministic logic depending on the operation/design. Resolve duplicates explicitly before MERGE. Do not advance an upstream checkpoint before the target commit is durable and validated - M08's candidate-checkpoint pattern still applies. Recovery: Idempotency is end-to-end. A perfect MERGE cannot save a pipeline that reads a moving, nondeterministic source on every retry. Preserve source evidence or a durable batch boundary; validate source key uniqueness; apply deterministic precedence; commit target; reconcile; only then commit external progress. This sequence connects ingestion state, transformation state and table state into one recoverable workflow.

Changed retry - do this without looking back

Change the source so two rows share one order_id with different updated_at values. Design a deterministic staging rule that surfaces exact ties as an error rather than arbitrarily ranking them - reuse the ambiguity discipline from M09.

Review 8 | DE13-08 - Slowly Changing Dimension Type 2 with Delta

Worked recovery

Customer C2 changes city from Patna to Pune effective 2026-09-14. Expire the old C2 row by setting `effective_to` to the new effective time and `is_current=false`; insert a new C2 row with city Pune, `effective_from=2026-09-14`, open-ended `effective_to` and `is_current=true`. Facts dated before the change can still join to the historical C2 version using the validity interval. A dashboard doubles customer counts after an SCD load. Investigation finds two `is_current=true` rows for some customer_ids because the expire step failed but inserts continued. Treat this as a table invariant failure, not a dashboard bug. Quarantine/repair the affected load, prove one current row per key, inspect temporal overlaps, and rerun with transactional/staged logic.

What a strong answer must prove

- Uses the lesson's business grain/key rather than only an API name.
- Separates physical files from logical table state and cites version/history when relevant.
- Includes an independent reconciliation, not just successful execution.
- States a realistic failure mode and the first evidence to inspect.
- Does not claim live Delta runtime if only static/simulator evidence exists.

Debugging path

Do not update the old row's attributes in place; that destroys history. Do not insert a new current version without expiring the old one; that produces two current rows. Do not compare every technical column when detecting change - audit timestamps can create a new version every run. Define the tracked business attributes deliberately. Recovery: SCD2 is a model, not a Delta feature. Delta supplies transaction and merge mechanics; you supply dimensional meaning. In a full warehouse, facts may store surrogate keys resolved at load time, or joins may use business key plus event timestamp against validity intervals. Both require clean temporal intervals. Your module goal is not to memorize one clever MERGE statement but to reason from invariants and validate them after every load.

Changed retry - do this without looking back

Now receive two changes for the same customer in the same source batch with different effective times. Explain why the simple one-row-per-key staging contract rejects this batch and what preprocessing/ordering logic would be required before SCD2 application.

Review 9 | DE13-09 - Bronze, Silver and Gold lakehouse layers

Worked recovery

If Bronze has 6 rows, one with negative quantity, Silver may contain 5 valid rows and quarantine 1 rejected row. Gold daily revenue should aggregate the 5 valid rows only. Your control equation is $6 = 5 + 1$, plus $\text{sum}(\text{Silver.gross_sales}) = \text{sum}(\text{Gold.daily_sales})$. That simple reconciliation is more trustworthy than saying 'the notebook ran green.' A Gold KPI is 4% lower than source finance totals. Start at Gold-to-Silver reconciliation, then Silver-to-Bronze/quarantine, then Bronze-to-source manifest. You may discover an intentional quality rejection, a duplicate rule, a late source batch or a Gold filter. Layering helps because each boundary has a contract and control total; without that, engineers jump randomly across notebooks.

What a strong answer must prove

- Uses the lesson's business grain/key rather than only an API name.
- Separates physical files from logical table state and cites version/history when relevant.
- Includes an independent reconciliation, not just successful execution.
- States a realistic failure mode and the first evidence to inspect.
- Does not claim live Delta runtime if only static/simulator evidence exists.

Debugging path

Do not transform Bronze so heavily that original source evidence is lost. Do not let Silver silently drop rows without a reconciliation path. Do not build Gold directly from raw sources when Silver defines the canonical cleaning/conformance rules unless the exception is deliberate and documented. And do not assume Bronze/Silver/Gold means three copies of every dataset forever; retention and design depend on requirements. Recovery: Medallion design is strongest when paired with idempotent rebuild rules. If Silver can be deterministically rebuilt from immutable/preserved Bronze evidence, incident recovery becomes simpler. Gold can then be rebuilt from stable Silver. This is not always free - large histories cost storage/compute - so production designs balance replay window, retention and recovery objectives. Keep the course pattern simple and explicit before adding streaming/change-data-feed complexity in M14.

Changed retry - do this without looking back

Now add a GDPR deletion requirement. Explain which layers and historical copies may be affected and why a simple 'delete from Gold' is not a complete governance response.

Review 10 | DE13-10 - Compaction, OPTIMIZE, VACUUM and maintenance boundaries

Worked recovery

A table has 2,000 tiny files from micro-batches. Compaction may replace them with fewer larger files in a new table version while keeping the same rows. Old files can remain physically present until VACUUM eligibility. If you immediately measure raw directory size, it may temporarily increase even though current query layout improved. Distinguish current active files from retained historical files. Storage cost spikes after a series of overwrites. An engineer proposes aggressive VACUUM. First inspect history, active file metrics, retention policy, longest-running readers/streams and recovery requirements. Maybe old files are legitimately retained; maybe compaction is needed; maybe the pipeline rewrites too much. Choose maintenance only after identifying the cause.

What a strong answer must prove

- Uses the lesson's business grain/key rather than only an API name.
- Separates physical files from logical table state and cites version/history when relevant.
- Includes an independent reconciliation, not just successful execution.
- States a realistic failure mode and the first evidence to inspect.
- Does not claim live Delta runtime if only static/simulator evidence exists.

Debugging path

Do not equate OPTIMIZE with removing duplicates; it changes physical layout, not business grain. Do not use VACUUM as a performance fix for bad query plans. Do not disable retention checks or set zero-hour retention in a beginner lab. Do not run maintenance during an unresolved corruption incident when historical versions may be needed for comparison/recovery. Recovery: Maintenance has three separate objectives: read performance/layout, storage cleanup and historical recoverability. They can conflict. A platform may optimize layout frequently but vacuum conservatively. Record maintenance operations in operational evidence and monitor their metrics. In cloud object storage, file count can affect listing/metadata overhead even when total bytes are modest; in local training, we simulate the reasoning at small scale without pretending it benchmarks production.

Changed retry - do this without looking back

Change the system so a streaming consumer can lag for five days. Explain how that changes retention thinking and why a generic '7 days is always enough' rule is not engineering reasoning.

Review 11 | DE13-11 - Integrated lakehouse pipeline and debugging checkpoint

Worked recovery

Suppose the second run produces one extra curated row. Work backward: Did source batch identity change? Did the staging query duplicate a key? Did MERGE use the stable business key? Did a Bronze append occur twice but Silver dedupe fail? Which reconciliation first diverged? The fastest debugger follows controls, not intuition. Table history tells you what committed; source/run metadata tells you why the job attempted it. At 08:00 Gold revenue is wrong. Bronze count matches source manifest. Silver valid + quarantine matches Bronze, but Silver revenue is too high. That narrows the failure to Silver transformation/dedupe rather than ingestion. Inspect key uniqueness and merge staging before touching Gold. If the wrong Silver commit is isolated and history retained, use time travel to compare versions and build a corrected forward commit/recovery plan.

What a strong answer must prove

- Uses the lesson's business grain/key rather than only an API name.
- Separates physical files from logical table state and cites version/history when relevant.
- Includes an independent reconciliation, not just successful execution.
- States a realistic failure mode and the first evidence to inspect.
- Does not claim live Delta runtime if only static/simulator evidence exists.

Debugging path

Do not call the checkpoint complete because every script exited zero. Deliberately break one case: duplicate source key, invalid amount, stale update or schema drift. Save the first failure, diagnose it, repair one cause, rerun and prove reconciliation. Do not hide the error by dropDuplicates without a deterministic rule or by overwriting the whole table. Recovery: A production lakehouse is an operational system, not a storage slogan. The strongest habit from M13 is to treat every table change as a state transition with evidence: intended input, deterministic transformation, transactional commit, independent reconciliation, history/version capture and safe progress. M14 will add streaming/event-driven behavior, where the same state discipline becomes even more important because retries and continuous processing amplify weak assumptions.

Changed retry - do this without looking back

Inject two conflicting source rows for the same key with the same updated_at but different amounts. The correct outcome is to surface ambiguity and stop before target progress. Explain why choosing whichever row appears last would make retries/order changes unsafe.